PDF Loading Using pypdf and LangChain Core

1. Quick Revision

In the previous session, we learned how to load information from text files.

We also created our own LangChain-compatible text loader using:

BaseLoader

Previously, we discussed:

PyPDFLoader

from the sunset:

langchain-community

For our revised course, we will use a more maintainable approach:

pypdf

+

langchain-core

2. Today's Master Formula

PDF

↓

pypdf.PdfReader

↓

Page Text

↓

LangChain Document

More clearly:

PDF File

↓

PdfReader

↓

Extract Text

↓

Document

↓

page_content + metadata

3. Why Do We Need a PDF Parsing Library?

A PDF is different from a normal text file.

A PDF may contain:

- Multiple pages
- Images
- Tables
- Fonts
- Positioned text
- Formatting

Therefore, we need a library that understands the PDF structure.

For today's class, we use:

pypdf

4. What Is PdfReader?

PdfReader is a class from the:

pypdf

package.

It allows us to:

Open PDF

↓

Inspect Pages

↓

Access Individual Pages

↓

Extract Text

Important import:

from pypdf import PdfReader

5. Very Important Clarification

PdfReader is **not** a LangChain class.

Its responsibility is:

PDF Processing

After extracting the text, we use:

Document

from LangChain Core to represent the extracted information.

So:

pypdf

↓

Extracts PDF Information

LangChain Document

↓

Represents Extracted Information

6. Two PDF Loading Approaches

We can design our own PDF loading behaviour.

Approach 1 — Page-Wise Loading

PDF

↓

Page 1 → Document 1

Page 2 → Document 2

Page 3 → Document 3

One PDF page becomes one Document.

Approach 2 — Whole-PDF Loading

PDF

↓

Page 1

Page 2

Page 3

↓

Combined Text

↓

One Document

All page text becomes one Document.

We control this behaviour in our own program.

7. Important Note About Scanned PDFs

This approach depends on extractable PDF text.

For scanned PDFs:

PDF Page

↓

Image

↓

No Normal Text Layer

basic pypdf text extraction may not work properly.

In such cases, we may need:

OCR

OCR means:

Optical Character Recognition

8. Important Syntax Before Programming

Requirement	Syntax
Open PDF	PdfReader(pdf_path)
Get page count	len(reader.pages)
Access first page	reader.pages[0]
Extract page text	page.extract_text()
Create Document	Document(page_content=..., metadata=...)
Create custom PDF loader	Inherit from BaseLoader
Return Documents through common API	loader.load()
Iterate page Documents	loader.lazy_load()

9. Required Installation

For these classroom programs:

python -m pip install pypdf reportlab

We use:

pypdf

for:

Reading PDFs

and:

reportlab

for:

Creating Sample PDFs

10. Why Are We Using ReportLab?

For classroom demonstrations, we want to create small sample PDF files ourselves.

For this we use:

from reportlab.pdfgen import canvas

Simple flow:

ReportLab

↓

Create PDF

↓

pypdf

↓

Read PDF

↓

LangChain Document

11. Program 1 — Read a Single-Page PDF

Objective

Create a single-page PDF, read it using PdfReader, and convert its extracted text into a LangChain Document.

```
from pathlib import Path


from reportlab.pdfgen import canvas

from pypdf import PdfReader


from langchain_core.documents import Document



# ========================================

# PREPARE SAMPLE PDF

# ========================================


pdf_path = "library_notice.pdf"



pdf = canvas.Canvas(
```

```
    pdf_path
)


pdf.drawString(
    60,
    750,
    "LIBRARY NOTICE"
)


pdf.drawString(
    60,
    720,
    "Library opens at 8 AM."
)


pdf.drawString(
    60,
```

```
    690,
    "Library closes at 6 PM."
)


pdf.save()


# ========================================
# READ PDF
# ========================================

reader = PdfReader(
    pdf_path
)


# ========================================
# CREATE LANGCHAIN DOCUMENT
# ========================================
```

```
page = reader.pages[0]



document = Document(
    page_content=
        page.extract_text() or "",

    metadata={
        "source": pdf_path,
        "source_type": "pdf",
        "page": 0,
        "total_pages":
            len(reader.pages)
    }
)



# ========================================
# DISPLAY RESULTS
```

```
# ========================================

print(
    "TOTAL PDF PAGES:",
    len(reader.pages)
)



print(
    "DOCUMENT TYPE:",
    type(document).__name__
)



print(
    "\nPAGE CONTENT:"
)

print(
    document.page_content
```

```
)



print(
    "\nPAGE INDEX:",
    document.metadata["page"]
)



print(
    "SOURCE:",
    document.metadata["source"]
)
```

12. Program 1 Flow

Create PDF

↓

library_notice.pdf

↓

PdfReader

↓

reader.pages[0]

↓

page.extract_text()

↓

Document

|

├── page_content

└── metadata

13. Understanding canvas.Canvas()

pdf = canvas.Canvas(

pdf_path

)

This creates a ReportLab Canvas object.

The Canvas is used to generate the PDF.

Conceptually:

canvas.Canvas()

↓

PDF Writing Object

↓

Add Content

↓

Save PDF

14. Understanding drawString()

Example:

```
pdf.drawString(
   60,
   750,
   "LIBRARY NOTICE"
)
```

The method takes three important arguments:

Argument	**Meaning**
60	X-coordinate
750	Y-coordinate
"LIBRARY NOTICE"	Text to display

15. Understanding X and Y Coordinates

X-coordinate

↓

Distance from Left

Y-coordinate

↓

Distance from Bottom

Example:

```
pdf.drawString(
   60,
   750,
   "LIBRARY NOTICE"
)
```

means:

60 points from left

750 points from bottom

Important:

72 points = 1 inch

16. Understanding pdf.save()

pdf.save()

finalises and writes the PDF file.

Concept:

Canvas

↓

Add Text

↓

save()

↓

PDF File Created

17. Understanding PdfReader

```
reader = PdfReader(
   pdf_path
)
```

This opens the PDF for reading.

Then:

reader.pages

provides access to PDF pages.

18. Accessing the First Page

page = reader.pages[0]

Important:

First Page

↓

Index 0

PDF page access here uses zero-based indexing.

19. Extracting Page Text

page.extract_text()

extracts available text from the page.

We use:

page.extract_text() or ""

Why?

Because if:

page.extract_text()

returns no useful text value, we use:

""

instead.

20. Creating the LangChain Document

```
document = Document(
    page_content=
        page.extract_text() or "",


    metadata={
        "source": pdf_path,
        "source_type": "pdf",
        "page": 0,
        "total_pages":
            len(reader.pages)
    }
)
```

The extracted text goes to:

page_content

and PDF information goes to:

metadata

21. Program 1 Metadata

Example:

```
{
   "source":
      "library_notice.pdf",

   "source_type":
      "pdf",

   "page":
      0,

   "total_pages":
      1
}
```

Memory:

source

↓

Which File?

source_type

↓

What Type?

page

↓

Which Page?

total_pages

↓

How Many Pages?

22. Program 2 — Multipage PDF Loading

Objective

Create a three-page PDF and convert each page into a separate LangChain Document.

23. Important ReportLab Methods

1. canvas.Canvas()

canvas.Canvas(pdf_path)

Creates a Canvas for generating the PDF.

2. drawString()

pdf.drawString(...)

Adds text to the current page.

3. showPage()

pdf.showPage()

Finishes the current page and starts the next page.

4. save()

pdf.save()

Finalises and writes the PDF to disk.

24. Program 2 — Complete Code

```
from pathlib import Path

from reportlab.pdfgen import canvas
from pypdf import PdfReader

from langchain_core.documents import Document
```

```
# ========================================
# CREATE THREE-PAGE PDF
# ========================================

pdf_path = (
    Path(__file__).resolve().parent
    / "company_policy.pdf"
)


pages = [
    (
        "ATTENDANCE POLICY",
        "Employees must report by 9 AM."
    ),

    (
        "LEAVE POLICY",
        "Employees receive 12 casual leaves."
```

```
    ),

    (
        "EMPLOYEE BENEFITS",
        "Medical insurance is available."
    )
]


pdf = canvas.Canvas(
    str(pdf_path)
)


for heading, content in pages:

    pdf.drawString(
        60,
        750,
        heading
```

```
    )

    pdf.drawString(
        60,
        720,
        content
    )

    pdf.showPage()


pdf.save()


# ========================================
# READ PDF AND CREATE DOCUMENTS
# ========================================

reader = PdfReader(
    str(pdf_path)
```

```
)


documents = []


for page_index, page in enumerate(
    reader.pages
):

    documents.append(
        Document(
            page_content=
                page.extract_text() or "",

            metadata={
                "source":
                    str(pdf_path),

                "page":
```

```
                page_index,

            "total_pages":
                len(reader.pages),

            "source_type":
                "pdf"
        }
    )
)


# ========================================
# DISPLAY RESULTS
# ========================================

print(
    "TOTAL DOCUMENTS:",
    len(documents)
)
```

```
for document in documents:

    page_number = (
        document.metadata["page"]
        + 1
    )


    text = " ".join(
        document.page_content.split()
    )


    print(
        "\nPAGE",
        page_number
    )
```

```
print(
    "CONTENT:",
    text
)
```

25. Creating Multiple Pages

We created:

```
pages = [
    (
        "ATTENDANCE POLICY",
        "Employees must report by 9 AM."
    ),
    (
        "LEAVE POLICY",
        "Employees receive 12 casual leaves."
    ),
    (
        "EMPLOYEE BENEFITS",
        "Medical insurance is available."
```

)

]

Each tuple contains:

Heading

+

Content

26. Understanding This Loop

for heading, content in pages:

Each tuple is unpacked.

For the first tuple:

heading

↓

ATTENDANCE POLICY

content

↓

Employees must report by 9 AM.

27. Why Use showPage()?

Inside the loop:

pdf.showPage()

means:

Finish Current Page

↓

Start Next Page

Without showPage(), all content may be written on the same page.

28. Reading Multiple PDF Pages

```
reader = PdfReader(
   str(pdf_path)
)
```

Then:

reader.pages

contains page objects.

We iterate using:

```
for page_index, page in enumerate(
   reader.pages
```

):

This gives:

page_index = 0 → First Page

page_index = 1 → Second Page

page_index = 2 → Third Page

29. Creating One Document Per Page

Inside the loop:

```
documents.append(
   Document(...)
)
```

Each PDF page becomes one Document.

Concept:

Page 1

↓

Document 1

Page 2

↓

Document 2

Page 3

↓

Document 3

30. Page-Wise Loading Formula

3 PDF Pages

↓

3 Page Objects

↓

3 Extracted Texts

↓

3 Documents

So:

len(documents)

returns:

3

31. Why Add 1 to Page Index?

Metadata contains:

document.metadata["page"]

which starts from:

0

For display:

```
page_number = (
   document.metadata["page"]
   + 1
)
```

So:

Index 0 → Page 1

Index 1 → Page 2

Index 2 → Page 3

32. Cleaning Extracted Text

We use:

```
text = " ".join(
    document.page_content.split()
)
```

This removes irregular whitespace.

Example:

```
"Python


    is     easy"
```

becomes:

Python is easy

33. Program 2 Flow

Create company_policy.pdf

↓

3 Pages

↓

PdfReader

↓

reader.pages

↓

Loop Through Pages

↓

extract_text()

↓

Document

↓

documents.append()

↓

List of Documents

34. Program 3 — Whole-PDF Loading

Objective

Create only one LangChain Document containing text from all PDF pages.

```
from pathlib import Path


from pypdf import PdfReader
```

```
from langchain_core.documents import Document


# ========================================
# SPECIFY EXISTING PDF
# ========================================

pdf_path = (
    Path(__file__).resolve().parent
    / "company_policy.pdf"
)


# ========================================
# READ ALL PDF PAGES
# ========================================

reader = PdfReader(
    str(pdf_path)
)
```

```
page_texts = [
    page.extract_text() or ""
    for page in reader.pages
]


# ========================================
# COMBINE PAGE TEXT
# ========================================

combined_text = (
    "\n--- PAGE BREAK ---\n"
    .join(page_texts)
)


# ========================================
# CREATE ONE LANGCHAIN DOCUMENT
```

```
# ========================================

document = Document(
    page_content=
        combined_text,

    metadata={
        "source":
            str(pdf_path),

        "source_type":
            "pdf",

        "total_pages":
            len(reader.pages),

        "loading_mode":
            "single"
    }
)
```

```
# ========================================
# DISPLAY RESULTS
# ========================================

documents = [
    document
]


print(
    "TOTAL DOCUMENTS:",
    len(documents)
)


print(
    "TOTAL PDF PAGES:",
    document.metadata[
```

```
        "total_pages"
    ]
)


print(
    "PAGE BREAKS:",
    document.page_content.count(
        "--- PAGE BREAK ---"
    )
)


print(
    "PAGE INDEX AVAILABLE:",
    "page" in document.metadata
)
```

35. Understanding page_texts

We use:

```
page_texts = [
    page.extract_text() or ""
    for page in reader.pages
]
```

This creates a list containing the extracted text from each page.

Conceptually:

```
[
    "Page 1 Text",
    "Page 2 Text",
    "Page 3 Text"
]
```

36. Understanding the List Comprehension

This:

```
page_texts = [
    page.extract_text() or ""
    for page in reader.pages
]
```

is conceptually similar to:

```
page_texts = []

for page in reader.pages:

    text = (
        page.extract_text()
        or ""
    )

    page_texts.append(
        text
    )
```

37. Combining All Page Text

We use:

```
combined_text = (
    "\n--- PAGE BREAK ---\n"
    .join(page_texts)
)
```

Suppose:

```
page_texts = [
    "Page 1",
    "Page 2",
    "Page 3"
]
```

Then:

```
Page 1

--- PAGE BREAK ---

Page 2

--- PAGE BREAK ---

Page 3
```

38. Why Use a Page Break Marker?

The marker:

--- PAGE BREAK ---

helps us identify where one page ends and the next begins.

Concept:

Combined Document

↓

Page 1 Text

↓

PAGE BREAK

↓

Page 2 Text

↓

PAGE BREAK

↓

Page 3 Text

39. Why Is There No page Metadata?

In page-wise loading:

One Document

↓

One Specific Page

So we can store:

"page": 0

But in whole-PDF mode:

One Document

↓

Contains All Pages

Therefore, the source program does not store a single page value.

Instead it stores:

"loading_mode": "single"

40. Page-Wise vs Whole-PDF Loading

Page-Wise	**Whole PDF**
One Document per page	One Document for whole PDF

Has page index	No single page index
Multiple Documents	One Document
Good for page-level tracking	Good for combined content
Metadata contains page	Metadata contains loading_mode="single"

Easy memory:

PAGE-WISE

↓

SEPARATE

WHOLE PDF

↓

COMBINE

41. Program 3 Flow

company_policy.pdf

↓

PdfReader

↓

All Pages

↓

extract_text()

↓

page_texts

↓

join()

↓

combined_text

↓

One Document

42. Understanding .count()

We use:

```
document.page_content.count(
   "--- PAGE BREAK ---"
)
```

This counts how many times the page-break marker appears.

If there are three pages:

Page 1

BREAK

Page 2

BREAK

Page 3

there are:

2 page-break markers

43. Program 4 — Custom PDF Loader Using BaseLoader

Objective

Combine:

pypdf

for PDF parsing

with:

BaseLoader

for a LangChain-compatible loader interface.

44. Complete Custom PDF Loader Program

```
from pathlib import Path

from pypdf import PdfReader

from langchain_core.documents import (
    Document
)

from langchain_core.document_loaders import (
    BaseLoader
)



# ======================================
# CUSTOM PDF LOADER
# ======================================

class PDFPageLoader(
    BaseLoader
```

```
):

    def __init__(
        self,
        pdf_path
    ):

        self.pdf_path = Path(
            pdf_path
        )


    def lazy_load(self):

        reader = PdfReader(
            str(self.pdf_path)
        )


        total_pages = len(
```

```
    reader.pages
)


for page_index, page in enumerate(
    reader.pages
):

    yield Document(
        page_content=
            page.extract_text()
            or "",

        metadata={
            "source":
                str(self.pdf_path),

            "source_type":
                "pdf",
```

```
                "page":
                    page_index,

                "total_pages":
                    total_pages
            }
        )


# ========================================
# USE CUSTOM LOADER
# ========================================

pdf_path = (
    Path(__file__).resolve().parent
    / "company_policy.pdf"
)


loader = PDFPageLoader(
```

```
    pdf_path
)


# ========================================
# PROCESS DOCUMENTS
# ========================================

count = 0


for document in loader.lazy_load():

    count += 1


    text = " ".join(
        document.page_content.split()
    )
```

```
    print(
        "PAGE:",
        document.metadata["page"]
        + 1
    )


    print(
        "STATUS:",
        (
            "TEXT FOUND"
            if text
            else
            "NO EXTRACTABLE TEXT"
        )
    )


print(
```

```
    "\nTOTAL PAGES PROCESSED:",
    count
)
```

45. Understanding the Custom Loader Class

```
class PDFPageLoader(
    BaseLoader
):
```

means:

PDFPageLoader

↓

Inherits

↓

BaseLoader

This makes our class follow the LangChain loader interface.

46. What Does the Loader Receive?

```
def __init__(
    self,
    pdf_path
```

):

The loader accepts:

pdf_path

which tells it which PDF to load.

Then:

self.pdf_path = Path(

    pdf_path

)

stores the path as a Path object.

47. Understanding lazy_load()

def lazy_load(self):

performs the actual loading.

Inside it:

reader = PdfReader(

    str(self.pdf_path)

)

opens the PDF.

Then:

for page_index, page in enumerate(

reader.pages

):

iterates over pages.

For every page:

yield Document(...)

produces one LangChain Document.

48. Custom Loader Internal Flow

PDFPageLoader

↓

lazy_load()

↓

PdfReader

↓

reader.pages

↓

Page 1

↓

yield Document 1

↓

Page 2

↓

yield Document 2

↓

Page 3

↓

yield Document 3

49. Why Use yield?

yield allows Documents to be produced one at a time.

yield Document 1

↓

Process

↓

yield Document 2

↓

Process

This is useful for:

lazy_load()

50. Detecting Whether Text Was Extracted

We clean the content:

```
text = " ".join(
   document.page_content.split()
)
```

Then:

"TEXT FOUND" if text

else "NO EXTRACTABLE TEXT"

checks whether meaningful extracted text exists.

Concept:

Extracted Text Available?

```
      / \
     Yes  No
     ↓    ↓
TEXT FOUND  NO EXTRACTABLE TEXT
```

51. Why Might There Be No Extractable Text?

Possible reasons include:

Blank Page

or:

Scanned Page

or a PDF where normal text extraction is not sufficient.

This does not necessarily mean the page contains no visible information.

52. Custom Loader Master Architecture

PDF FILE

|

↓

PDFPageLoader

|

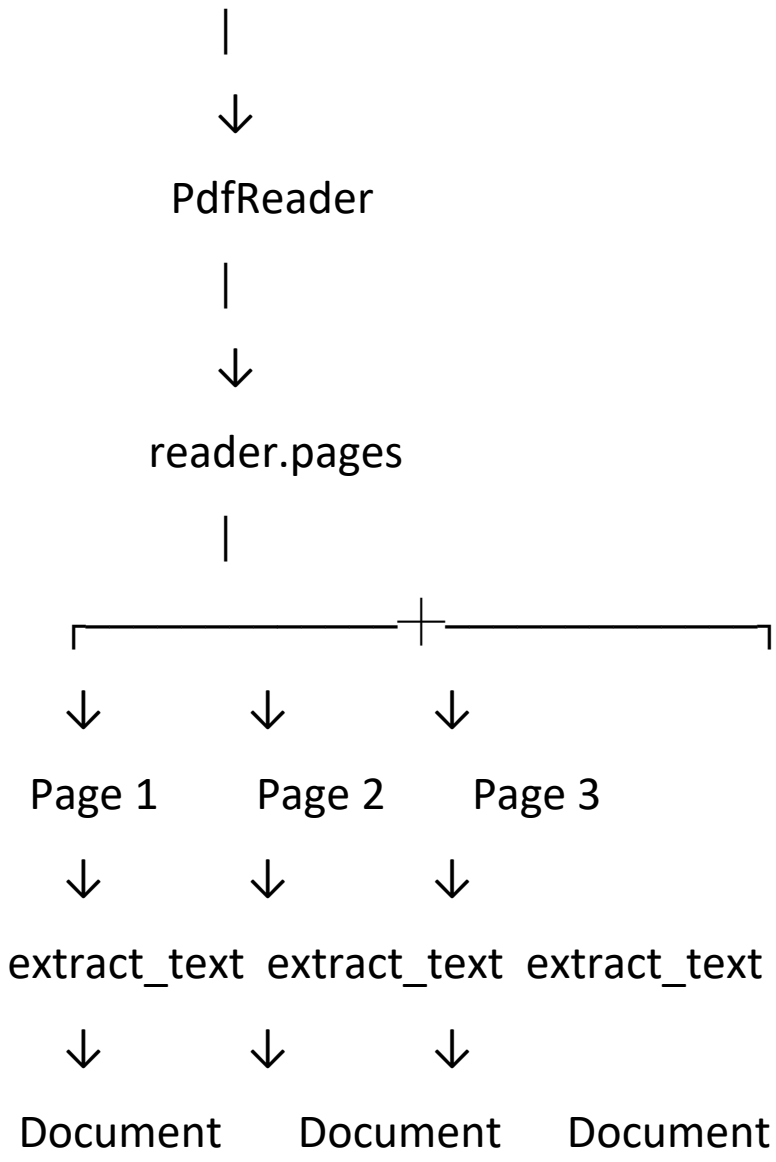
↓

BaseLoader

|

↓

lazy_load()



53. Why Is the Custom Loader Better Structured?

Instead of writing PDF extraction logic everywhere:

Application 1

↓

Custom PDF Code

Application 2

↓

Custom PDF Code

Application 3

↓

Custom PDF Code

we can create one reusable loader:

PDFPageLoader

↓

Reusable

↓

LangChain-Compatible

54. pypdf vs Document vs BaseLoader

These three have different responsibilities.

pypdf

PDF Processing

Document

Represent Extracted Information

BaseLoader

Provide Standard Loader Interface

Easy memory:

pypdf

↓

READ

Document

↓

REPRESENT

BaseLoader

↓

STANDARDIZE

55. Earlier Approach vs Revised Approach

Earlier

PDF

↓

PyPDFLoader

↓

langchain_community

↓

Document

Revised Course Approach

PDF

↓

pypdf

↓

Custom BaseLoader

↓

langchain_core Document

56. Important Conclusions

1. pypdf extracts text from PDF pages.
2. LangChain Document represents the extracted information.
3. Page-wise loading can create one Document per PDF page.
4. Whole-document loading combines multiple pages into one Document.
5. Metadata should preserve source and page information where applicable.

6. The first page index is 0.
7. Basic text extraction does not guarantee successful OCR.
8. BaseLoader provides the interface used here for custom PDF loaders.

57. Common Mistake — Thinking PdfReader Is a LangChain Class

Wrong:

PdfReader

↓

LangChain Class

Correct:

PdfReader

↓

pypdf Class

LangChain enters after extraction through:

Document

and optionally:

BaseLoader

58. Common Mistake — Thinking One PDF Means One Document

Not necessarily.

Page-wise design:

3 PDF Pages

↓

3 Documents

Whole-PDF design:

3 PDF Pages

↓

1 Document

The behaviour depends on our program.

59. Common Mistake — Thinking Page Index Starts from 1

Wrong:

First Page

↓

page = 1

Correct:

First Page

↓

page = 0

For normal display:

page_number = (

document.metadata["page"]

+ 1

)

60. Common Mistake — Thinking extract_text() Always Works

Not always.

For example:

Scanned PDF

↓

Page Is Mainly Image

↓

Basic extract_text()

May Return Little or No Text

Such cases may require OCR.

61. Common Mistake — Forgetting showPage()

When creating a multipage PDF with ReportLab:

pdf.showPage()

finishes the current page and starts another page.

Without it, the intended multipage structure may not be created.

62. Common Mistake — Forgetting pdf.save()

After drawing PDF content:

pdf.save()

must be called to finalise and write the PDF.

63. Four Programs — Quick Revision

Program 1

Single-Page PDF

↓

PdfReader

↓

One Document

Program 2

Multipage PDF

↓

One Document Per Page

Program 3

Multipage PDF

↓

Combine Page Text

↓

One Document

Program 4

pypdf

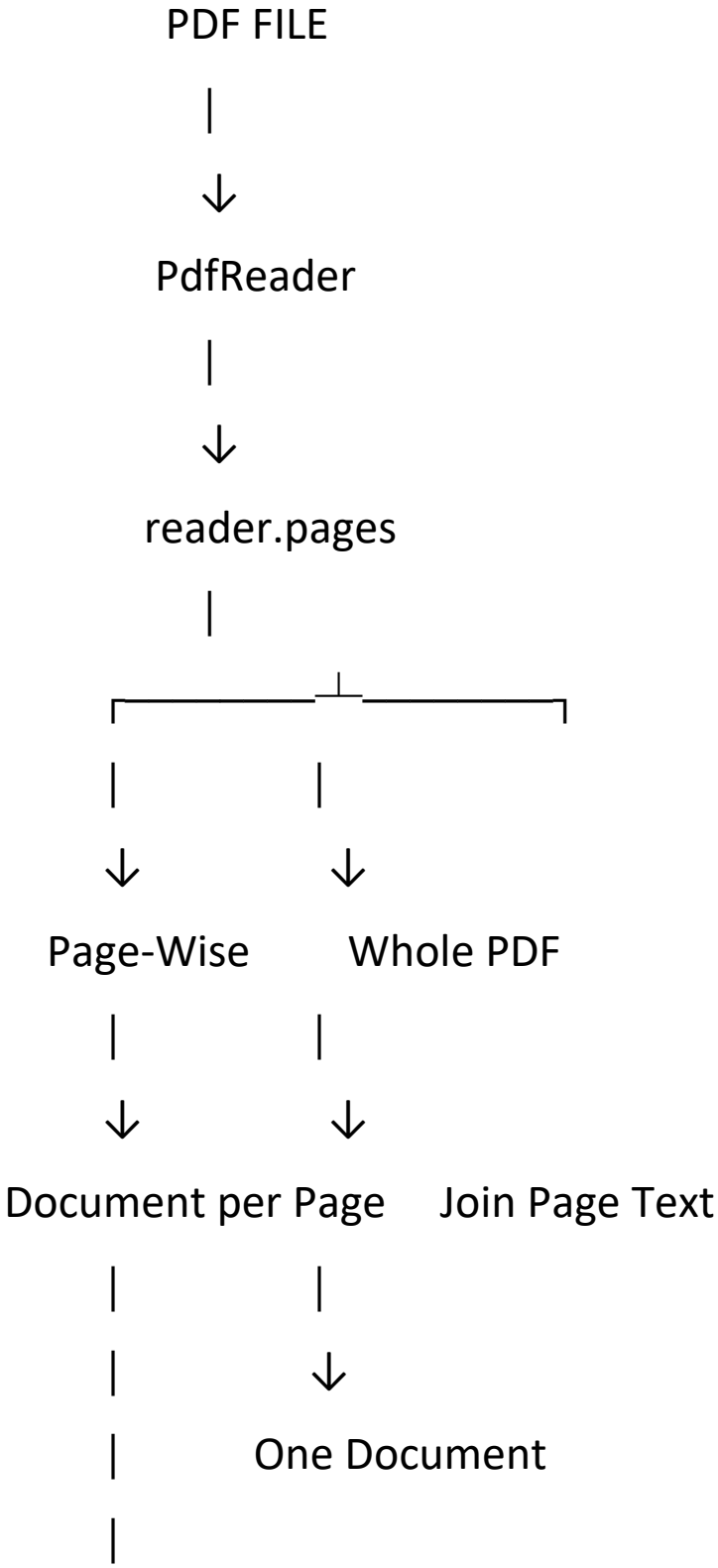
+

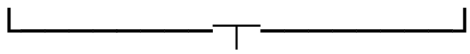
BaseLoader

↓

Reusable Custom PDF Loader

64. Today's Master Architecture





↓

LangChain Core

↓

Document

65. Final Master Formula

PDF

↓

pypdf

↓

Extract Text

↓

LangChain Document

For reusable loading:

PDF

↓

pypdf

↓

Custom BaseLoader

↓

lazy_load()

↓

Document Objects

66. One-Line Conclusion

Use pypdf to parse and extract text from PDF pages, use LangChain Core Document objects to represent that information, and use BaseLoader when we want a reusable LangChain-compatible PDF-loading interface.